CareerTrack

Minor Project 6 | Azure Functions

CSCE 547 Windows Programming

Github Repository: https://github.com/dswain07/CareerTrack---Azure-Functions

Student: Divyashanu Swain

Date: November 9th, 2026

1. Project overview

Minor Project 6 extends the CareerTrack application from Project 5. The existing HTML/CSS/JavaScript client communicates with an ASP.NET Core Web API service layer, which calls CareerTrack.Business. Two new business-layer methods use HttpClient to invoke deployed Azure Functions. A separate Azure Blob Storage integration generates and stores JSON application reports as the additional graduate-level Azure feature.

Component	Implementation
Client	HTML, CSS, JavaScript (CareerTrack.Api/wwwroot)
Service layer	ASP.NET Core Web API — JobApplications controller
Business layer	JobApplicationService and BlobReportService
Azure Functions	CalculateApplicationScore; GenerateFollowUpRecommendation
Additional Azure feature	Private Azure Blob Storage container: careertrack-reports
Database	SQLite with Entity Framework Core repositories

2. Azure configuration and deployment

Question i What steps did you take to configure Azure?

- Registered for Azure for Students without supplying a credit card, and created the CareerTrack-MP6-RG resource group.
- Created a Standard, locally redundant Azure Storage account named careertrackmp6storage in West US 2 after an East US regional-policy restriction prevented provisioning.
- Initially attempted Windows Consumption hosting; deployment was blocked because the subscription had a Y1 App Service quota of zero, and a request to raise the quota was unsuccessful.

- Created careertrack-mp6-flex using Linux Flex Consumption, .NET 10 isolated worker, West US 2, and a 512 MB instance size. Reused the storage account; disabled optional Azure OpenAI, Application Insights, and continuous deployment.
- Published the CareerTrack.AzureFunctions project from Visual Studio 2026 and verified both cloud HTTP endpoints returned the expected JSON.
- Created the private careertrack-reports blob container and configured the API to access it using Azure Storage SDK and a locally stored connection string in User Secrets.

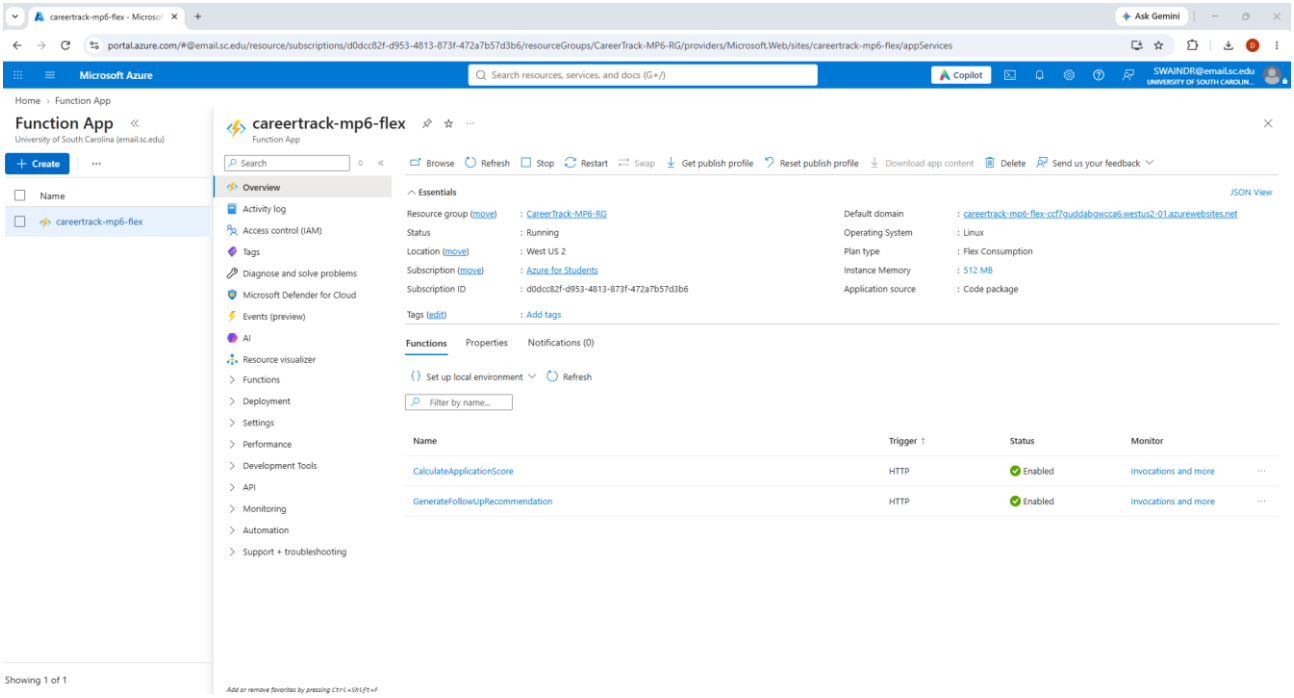


Figure 1. Azure Function App overview showing the running Flex Consumption deployment and both enabled HTTP triggered functions.

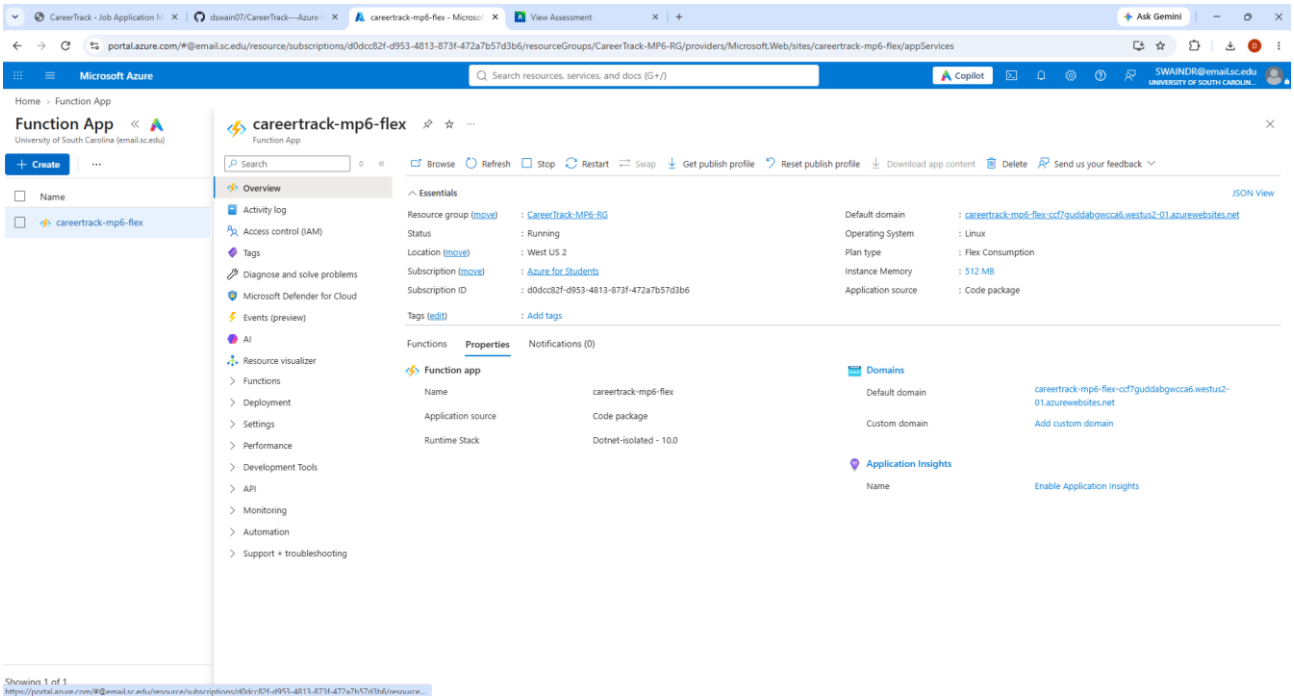


Figure 2. Azure Function App configuration showing the .NET isolated runtime and Linux Flex Consumption hosting environment.

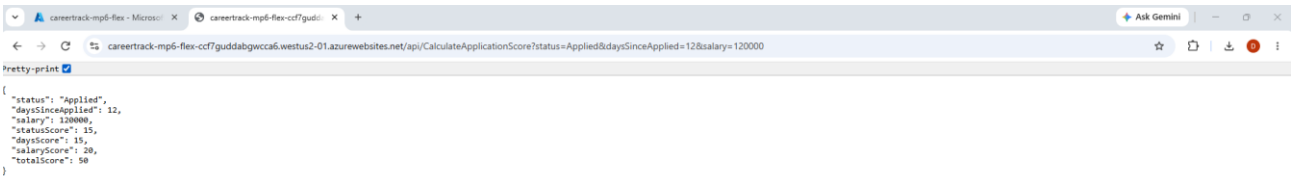


Figure 3. Successful execution of the deployed CalculateApplicationScore Azure Function, returning a total application score of 50.

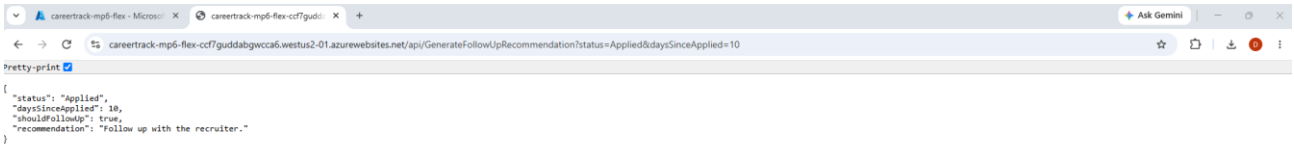


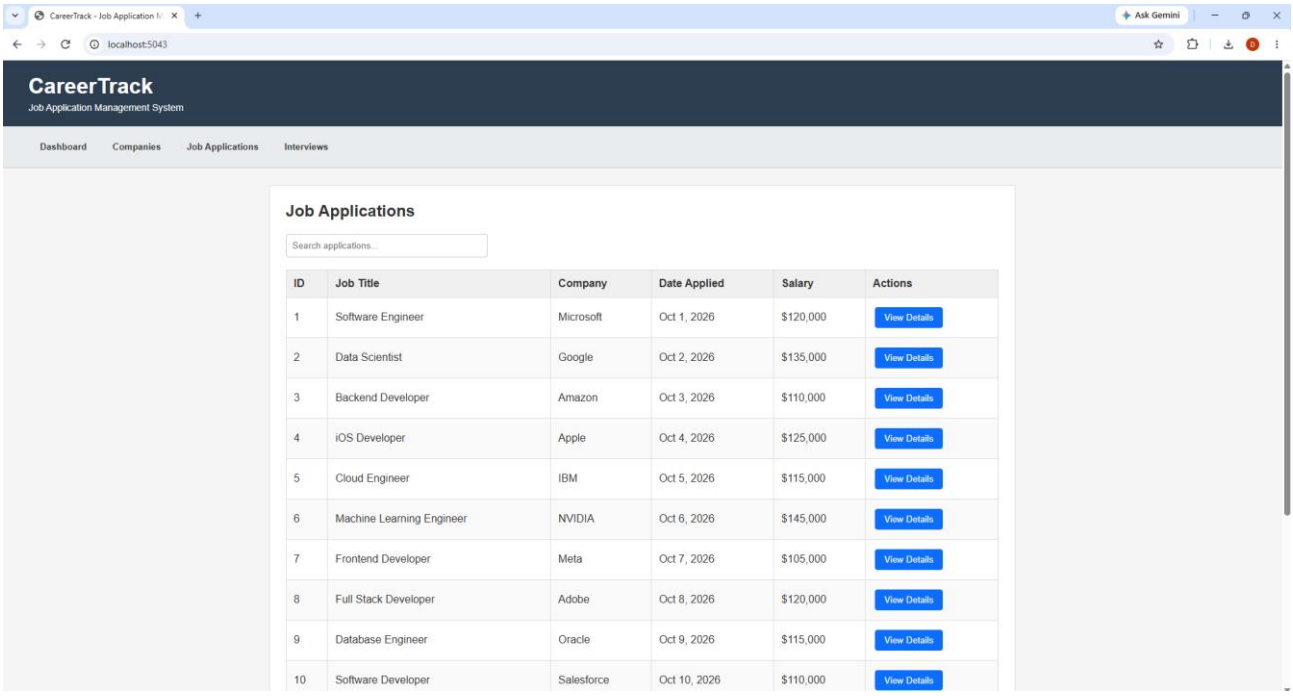
Figure 4. Successful execution of the GenerateFollowUpRecommendation Azure Function, recommending recruiter follow-up for an application submitted 10 days earlier.

3. Function implementation and application integration

The first HTTP-triggered function, CalculateApplicationScore, accepts status, daysSinceApplied, and salary, validates them, and returns a deterministic priority score. The second, GenerateFollowUpRecommendation, accepts status and elapsed days and returns a Boolean follow-up flag plus a recommendation message. These are rule-based demonstration features rather than predictive hiring models.

The client calls /api/jobapplications/application-score and /api/jobapplications/follow-up-recommendation using JavaScript fetch. The API controller calls the corresponding JobApplicationService methods, which use a configured HttpClient to invoke Azure. The client never calls the Azure Function endpoints directly.

Test	Observed result
Application score: Applied, 12 days, $120,000	totalScore = 50
Follow-up: Applied, 10 days	shouldFollowUp = true
End-to-end web interface	Calculate Score and Follow-Up buttons display results
Regression testing	Previously implemented features continued to work



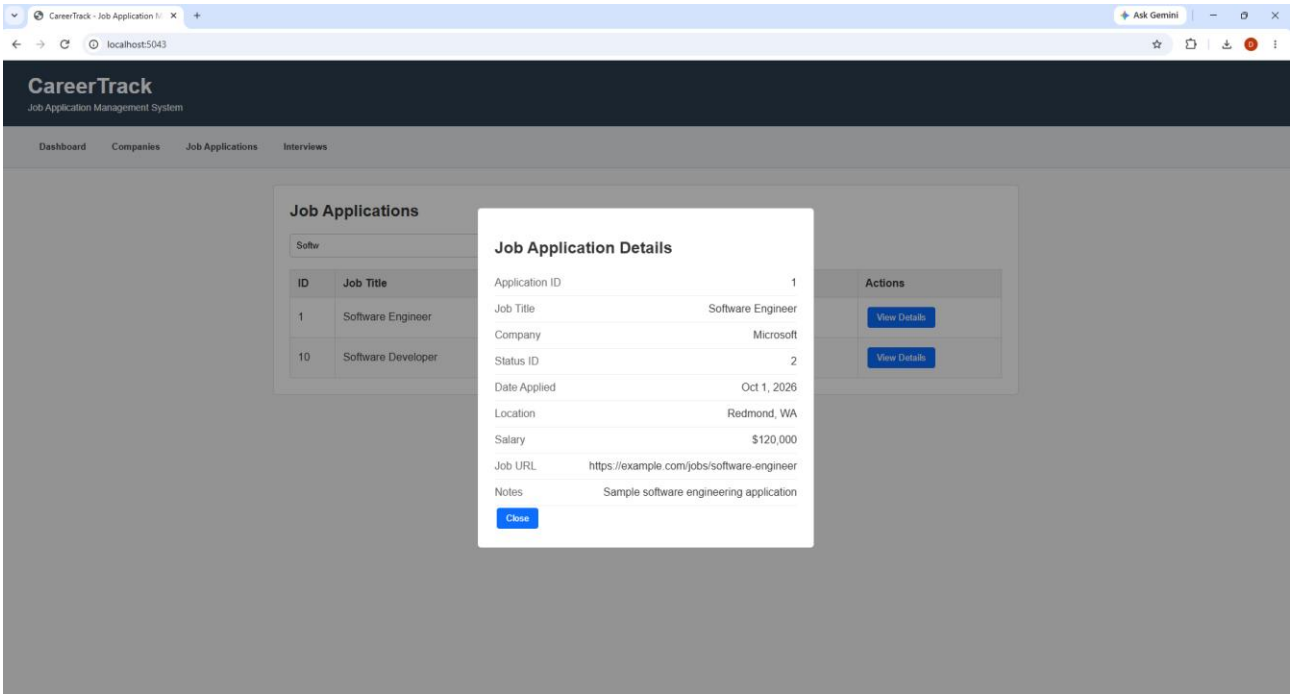


Figure 5 & 6. CareerTrack Job Applications interface from Minor Project 5, before Azure integration. The original interface provided application records and a View Details action.

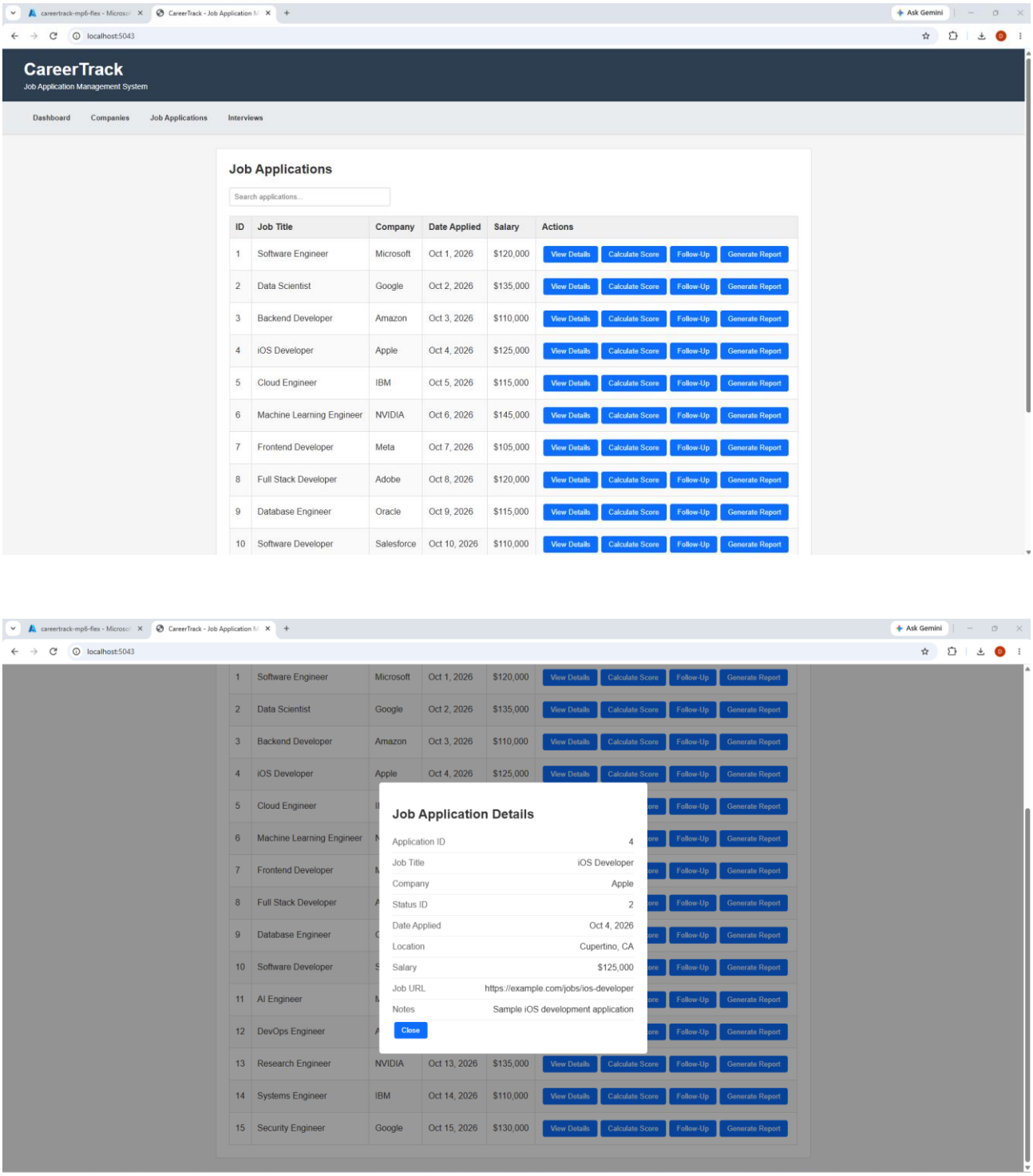


Figure 7 & 8. Updated CareerTrack Job Applications interface showing the three new Azure-integrated actions alongside the existing View Details functionality.

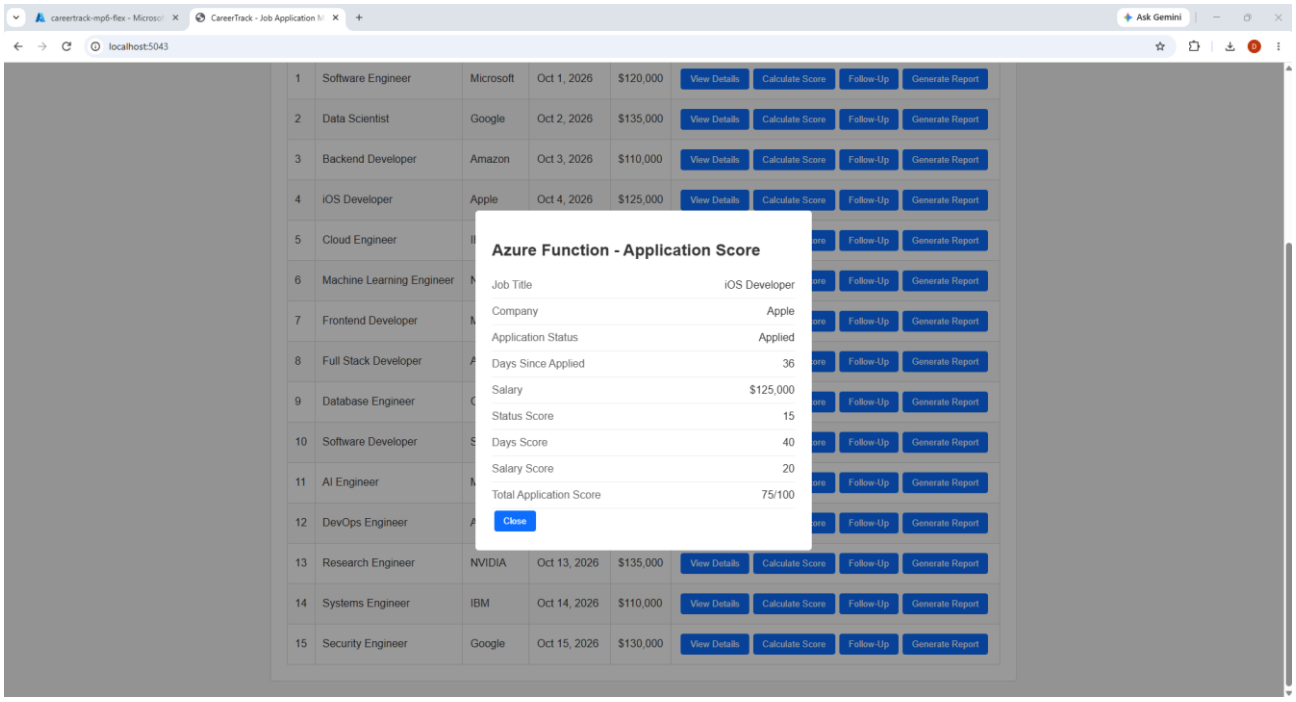


Figure 9. CareerTrack displaying the Azure Function application scoring result for an iOS Developer application, with a total score of 75/100.

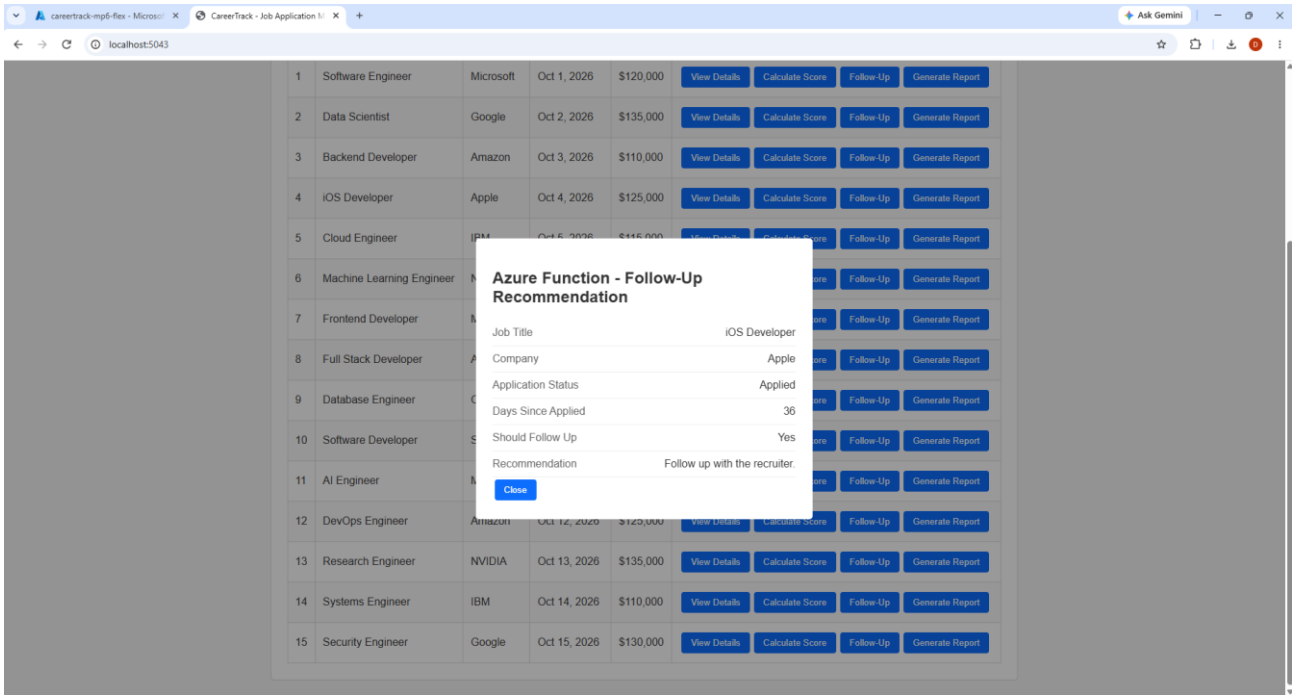


Figure 10. CareerTrack displaying a follow-up recommendation returned by the deployed Azure Function for an application submitted 36 days earlier.

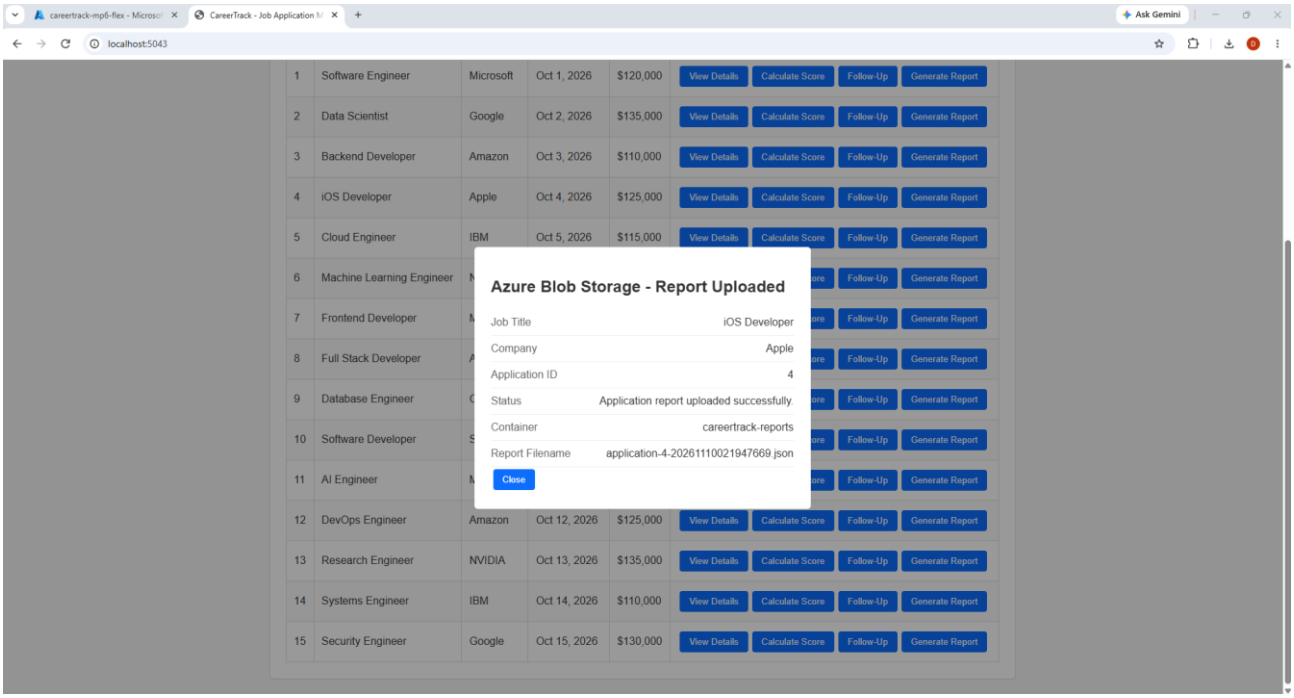


Figure 11. CareerTrack confirming the successful generation and upload of a JSON application report to the private Azure Blob Storage container.

4. Additional Azure feature

I implemented Azure Blob Storage as the required additional Azure feature. The careertrackmp6storage account contains a private container named careertrack-reports. The BlobReportService uses the Azure.Storage.Blobs SDK to retrieve a selected application and company from SQLite, serialize a small JSON report, and upload it as a timestamped blob. The API exposes POST /api/jobapplications/{id}/upload-report, and the web client offers a Generate Report button. The upload was verified using an API response, the client confirmation dialog, and the uploaded JSON blob in the Azure Portal.

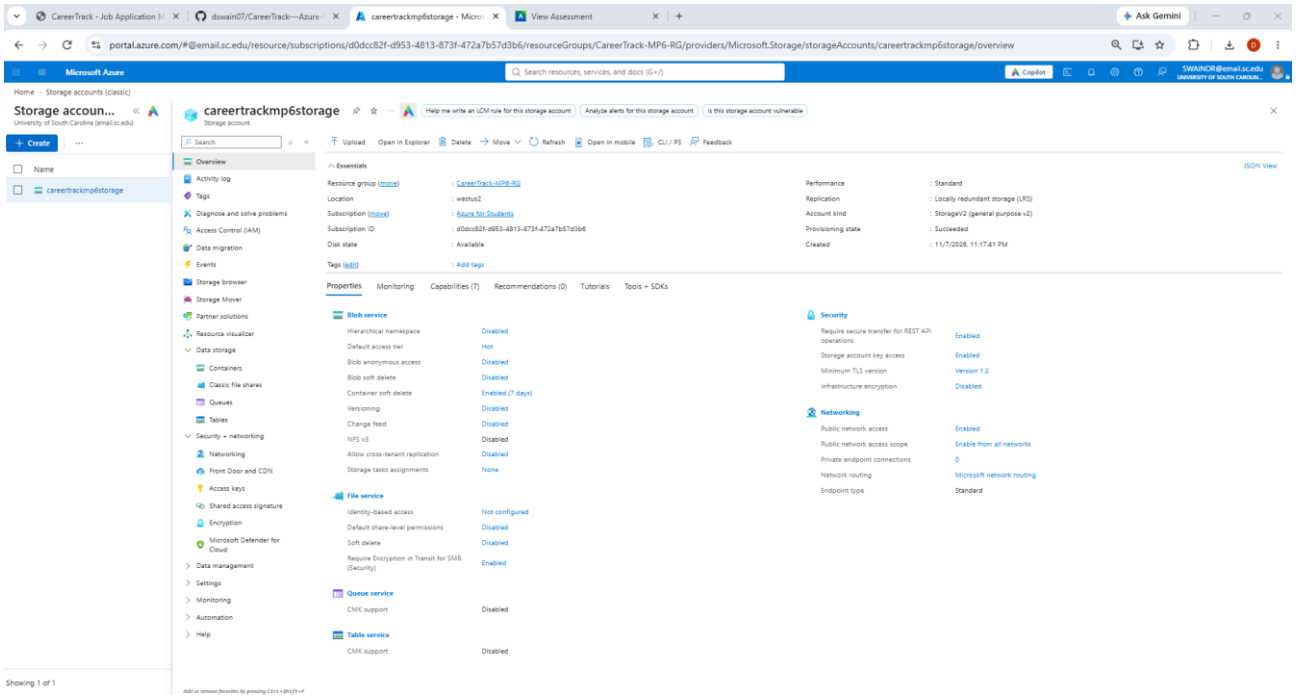


Figure 12. Azure Storage account configuration showing Standard performance, StorageV2 account type, locally redundant storage (LRS), West US 2 deployment region, and security settings used by CareerTrack.

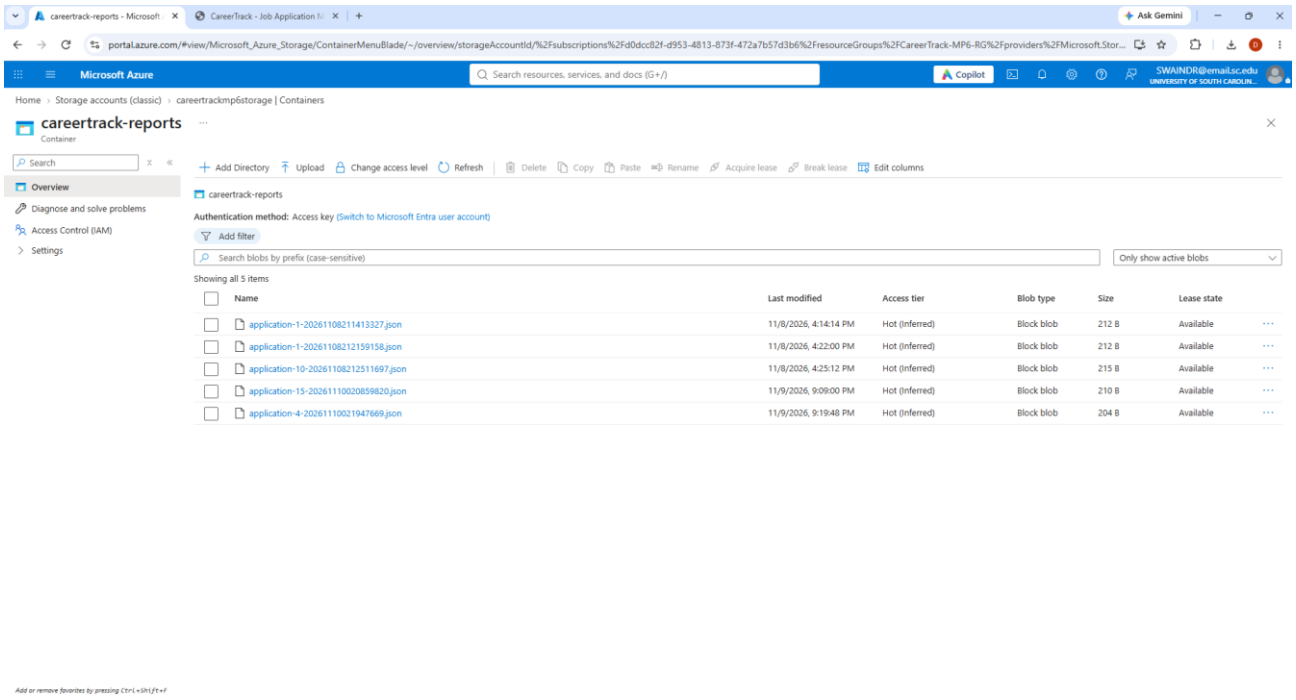


Figure 13. Azure Blob Storage container displaying five JSON application reports, including the report generated and uploaded through the CareerTrack web interface for application ID 4.

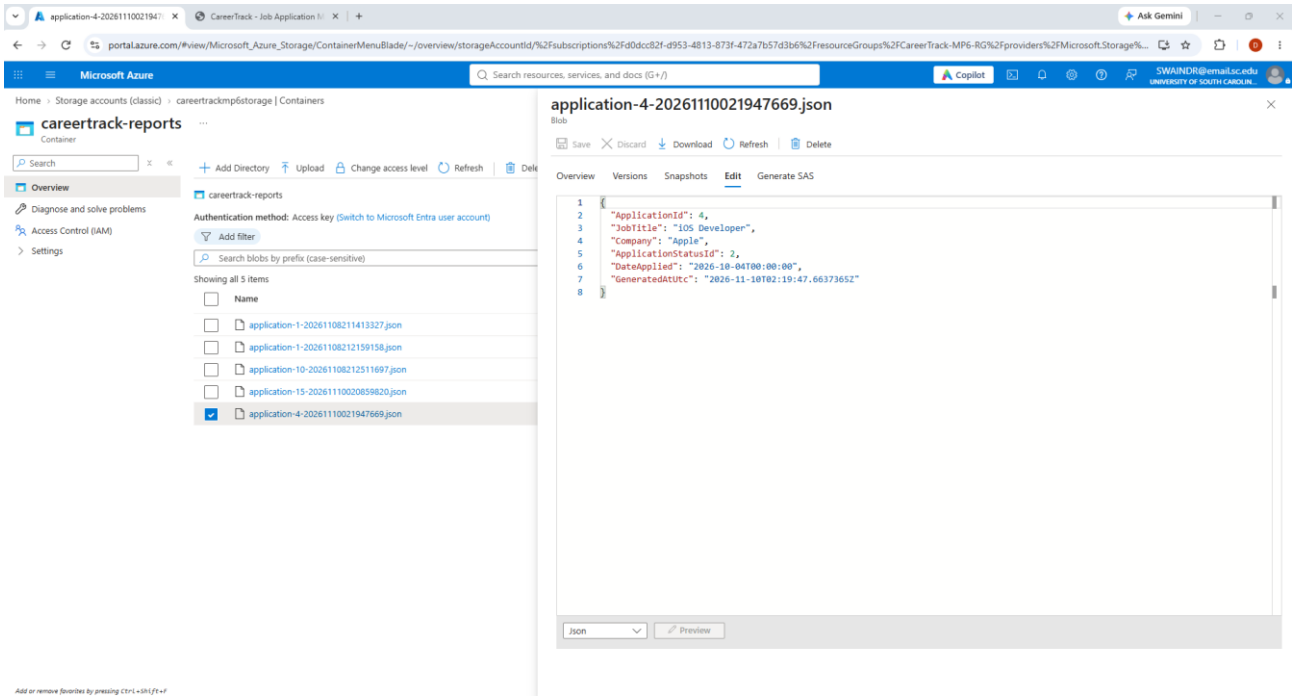


Figure 14. JSON report for Application ID 4 (iOS Developer at Apple), generated by CareerTrack and stored in Azure Blob Storage. The report contains application information and a UTC generation timestamp.

5. Reflection questions

ii. Describe a challenge you overcame in this project

The main challenge was deploying Azure Functions using the Azure for Students subscription. The initial storage deployment was blocked by an Azure policy limiting available regions. I inspected the allowed-region policy and successfully provisioned storage in West US 2. The Windows Consumption Function App then failed because the subscription had a Y1 App Service quota of zero. My quota increase request was unsuccessful, so I evaluated a different hosting model and successfully deployed using Flex Consumption. This taught me to distinguish source-code problems from cloud subscription policies, regional availability, and resource quotas.

iii. Describe a technique you learned in this project

I learned how to invoke serverless Azure Functions through a layered application using dependency injection and HttpClient. Instead of calling cloud endpoints directly from JavaScript, the client sends requests to ASP.NET Core controllers; those controllers call business-layer methods, which make HTTP requests to Azure and deserialize JSON responses into typed C# models. I also learned to use the Azure Storage SDK to upload JSON through a private blob container while keeping connection strings outside source control using User Secrets.

iv. What would you refine before selling, shipping, and deploying?

Before production release, I would add user authentication and role-based authorization, replace anonymous Azure Function access with protected endpoints, and use managed identities or a dedicated

secret-management service instead of storage keys where supported. I would introduce per-user data isolation, stronger request validation, structured error handling, rate limiting, audit logging, automated tests, and CI/CD. I would also move the local SQLite database to a production-ready managed database, configure monitoring and cost alerts, secure report downloads, and improve the application scoring rules through user testing. The current implementation is suitable as a coursework demonstration, not a production deployment.